

DATA SCIENCE & AI MASTERS 2026

DAY 4 - PYTHON OOP, ERROR HANDLING & EXCEPTIONS

Detailed Interview Question & Answer Bank

Scope	Object-Oriented Programming, Error Handling, Exceptions
Question count	135
Priority	40 ★ MUST ASK questions, grouped first
Purpose	Interview preparation for Data Science & AI roles
Coverage	Concepts, design, Python behavior, debugging, best practices

How to study these notes: Answer each question aloud first. Then compare with the reference answer. For coding/design questions, write a tiny example from memory. For ★ MUST ASK questions, aim to explain the concept, give an example, and mention one practical trade-off or failure mode.

Day 4 alignment: This document follows the Day 4 focus in your 120-day plan and uses the same interview-oriented scope: OOP, error handling, and exceptions.

★ MUST ASK - HIGH-PRIORITY QUESTIONS

Master these first. These questions are the most important for interviews because they test both Python knowledge and your ability to reason about design and failure handling.

1. ★ MUST ASK - What is object-oriented programming (OOP), and why is it useful in Python?

Answer: Object-oriented programming organizes software around objects that combine state (data) and behavior (methods). In Python, OOP is useful for structuring larger programs, improving reuse, separation of responsibilities, testing, and maintainability. In data science, classes can encapsulate preprocessing, model configuration, training, prediction, and validation logic.

2. ★ MUST ASK - What is the difference between a class and an object?

Answer: A class is a blueprint or definition describing attributes and behavior; an object is a concrete instance created from that class. For example, a class `Model` can define `fit()` and `predict()`, while `model = Model()` creates one object.

3. ★ MUST ASK - How do you create a class and instantiate an object in Python?

Answer: Define a class with the `class` keyword, then call the class like a function to create an instance. Example: `class Car: pass` followed by `c = Car()`. The call creates an object and invokes initialization behavior if defined.

4. ★ MUST ASK - What is an instance attribute versus a class attribute?

Answer: An instance attribute belongs to one particular object, while a class attribute is shared through the class unless an instance shadows it. Instance attributes are normally created with `self.x`; class attributes are defined directly in the class body.

5. ★ MUST ASK - What is `__init__()`, and when is it called?

Answer: `__init__()` initializes a newly created instance. It is called automatically during normal instance creation, after the object itself has been created. It is not the method that allocates the object; `__new__()` is involved in object creation.

6. ★ MUST ASK - What is `self` in a Python class, and why is it needed?

Answer: `self` is the conventional name for the current instance passed to an instance method. It lets the method access that object's attributes and other methods, such as `self.name` or `self.predict()`. Python does not make `self` a keyword; it is a convention.

7. ★ MUST ASK - What is an instance method? How is it different from a normal function?

Answer: An instance method is a function defined inside a class whose first parameter normally receives the instance. When accessed through an object, Python binds the object to that parameter automatically. A standalone function is not automatically bound to an instance.

8. ★ MUST ASK - What does encapsulation mean in Python? Does Python have truly private attributes?

Answer: Encapsulation means keeping data and the operations that manage it together and controlling how internal state is accessed. Python does not enforce Java-style private instance variables; `_name` is a convention, while `__name` triggers name mangling to reduce accidental access and collisions.

9. ★ MUST ASK - What is inheritance, and when would you use it?

Answer: Inheritance lets a child class reuse or extend behavior from a parent class. Use it when there is a genuine 'is-a' relationship and the child should conform to or specialize the parent's interface. Avoid it when composition gives a clearer design.

10. ★ MUST ASK - What is polymorphism in Python? Give a practical example.

Answer: Polymorphism means different objects can be used through a common interface even if their implementations differ. For example, different model classes may each implement `predict(X)`, allowing calling code to use `model.predict(X)` without caring which concrete model is supplied.

11. ★ MUST ASK - What is abstraction, and how can Python implement it?

Answer: Abstraction exposes the essential interface while hiding implementation details. Python can implement it with conventions, well-defined interfaces, duck typing, or the `abc` module using abstract base classes and `@abstractmethod` when explicit contracts are useful.

12. ★ MUST ASK - What is method overriding?

Answer: Method overriding occurs when a subclass defines a method with the same name as a method inherited from its parent, replacing or specializing the inherited behavior for that subclass.

13. ★ MUST ASK - What does `super()` do, and why is it useful?

Answer: `super()` gives access to behavior defined in the next class in the method resolution order. It is commonly used to call a parent/next implementation, especially in cooperative multiple inheritance, without hard-coding a specific parent class.

14. ★ MUST ASK - What are instance methods, class methods, and static methods?

Answer: Instance methods receive `self` and operate on an instance. Class methods use `@classmethod` and receive `cls`, so they operate on class-level state or act as alternative constructors. Static methods use `@staticmethod` and receive no automatic instance or class reference; they are namespace-related utility functions.

15. ★ MUST ASK - When should you use `@classmethod` instead of an instance method?

Answer: Use a class method when the operation logically belongs to the class rather than a specific existing instance, especially for alternative constructors or class-level configuration. Example: `Data.from_csv(path)` can construct and return a new instance.

16. ★ MUST ASK - When should you use `@staticmethod`?

Answer: Use a static method when a function belongs conceptually to the class's namespace but does not need `self` or `cls`. If there is no meaningful relationship to the class, a module-level function may be clearer.

17. ★ MUST ASK - What is `cls` in a class method?

Answer: `cls` is the conventional name for the class object passed automatically to a class method. It lets the method read or modify class attributes and instantiate the class in a subclass-friendly way.

18. ★ MUST ASK - What are dunder/magic methods, and why are they important?

Answer: Dunder methods are special methods with double underscores, such as `__init__`, `__str__`, and `__len__`. Python calls them implicitly for language operations, allowing custom objects to integrate naturally with built-in syntax and protocols.

19. ★ MUST ASK - What is the difference between `__str__()` and `__repr__()`?

Answer: `__str__()` aims to provide a readable, user-facing representation. `__repr__()` aims to provide a developer-oriented, unambiguous representation useful for debugging. `print(obj)` prefers `__str__`, while `repr(obj)` explicitly uses `__repr__`.

20. ★ MUST ASK - What is composition, and how does it differ from inheritance?

Answer: Composition means building a class from other objects that it contains or uses. Inheritance creates an 'is-a' relationship; composition usually creates a 'has-a' or 'uses-a' relationship. Composition often reduces coupling and makes behavior easier to change.

21. ★ MUST ASK - Explain the IS-A and HAS-A relationships with Python examples.

Answer: IS-A describes inheritance: Dog is-an Animal. HAS-A describes composition: Car has-an Engine. These relationships guide design decisions; use inheritance when substitutability is intended, and composition when one component simply uses another.

22. ★ MUST ASK - What is multiple inheritance, and what problems can it introduce?

Answer: Multiple inheritance lets a class inherit from more than one parent. It can be useful for orthogonal mixins, but may introduce ambiguous method lookup, diamond-shaped hierarchies, and tightly coupled designs if used carelessly.

23. ★ MUST ASK - What is Python method resolution order (MRO), and how does it affect multiple inheritance?

Answer: MRO is the order Python uses to search classes for methods and attributes. Python computes it using C3 linearization. You can inspect it with `Class.mro()` or `Class.__mro__`. It determines which implementation is found first in multiple-inheritance hierarchies.

24. ★ MUST ASK - What is duck typing, and how does it relate to polymorphism in Python?

Answer: Duck typing means code cares about what an object can do rather than its exact type: 'if it behaves like the required interface, use it.' This is a major source of Python's flexible polymorphism.

25. ★ MUST ASK - What is an exception, and how is it different from a syntax error?

Answer: An exception is a runtime event represented by an exception object that interrupts normal execution and can be handled. A syntax error means Python cannot parse the source code as valid Python syntax; it is detected before normal execution of that code.

26. ★ MUST ASK - How does try/except work in Python?

Answer: Code that may fail is placed in `try`; matching exceptions are handled by `except` blocks. If an exception occurs, Python skips the remaining `try` statements and searches for a matching handler. If no handler matches, the exception propagates upward.

27. ★ MUST ASK - Why should you catch specific exceptions instead of using bare except?

Answer: Specific catches preserve visibility into unexpected failures and prevent unrelated problems from being silently swallowed. `except ValueError:` is usually better than `except:` when only invalid input is expected.

28. ★ MUST ASK - What are the purposes of else and finally in exception handling?

Answer: `else` runs only if the `try` block completed without raising. `finally` runs as cleanup code whether or not an exception occurred. Keeping success-path logic in `else` and cleanup in `finally` improves clarity.

29. ★ MUST ASK - How do you raise an exception explicitly with raise?

Answer: Use `raise`, optionally with an exception instance or class, for example `raise ValueError('age must be positive')`. This is useful for enforcing preconditions and signaling invalid states.

30. ★ MUST ASK - How do you create a custom exception class, and when should you use one?

Answer: Subclass `Exception`, for example `class DataValidationError(Exception): pass`. Use custom exceptions when a domain-specific failure needs clearer meaning or selective handling, such as invalid training data or a model configuration error.

31. ★ MUST ASK - What is the relationship between BaseException and Exception?

Answer: `BaseException` is the root of Python's built-in exception hierarchy. `Exception` is a major branch intended for ordinary application/runtime errors. Many control-flow-oriented exceptions, such as `KeyboardInterrupt`, inherit directly from `BaseException` rather than `Exception`.

32. ★ MUST ASK - When will a finally block execute, and why is it useful for cleanup?

Answer: A `finally` block runs when execution leaves the `try` statement, including after normal completion and most exception paths. It is useful for cleanup such as closing resources or releasing locks.

33. ★ MUST ASK - What happens if a function returns inside try and finally also contains a return?

Answer: The `finally` block runs before the function returns. If `finally` also executes a `return`, that later `return` overrides the earlier `return` and can suppress an exception. This is why returning from `finally` is generally discouraged.

34. ★ MUST ASK - What is exception chaining, and what is the purpose of from in a raise statement?

Answer: Exception chaining preserves the relationship between a higher-level error and the original cause. `raise NewError('...') from e` explicitly records `e` as the cause, making layered debugging clearer.

35. ★ MUST ASK - How do you re-raise the currently handled exception?

Answer: Inside an except block, use bare raise. It re-raises the current exception while preserving its traceback. raise e is different and can affect traceback presentation.

36. ★ MUST ASK - What is a context manager, and why is it important for safe resource handling?

Answer: A context manager defines setup and cleanup behavior around a block of code, normally through __enter__ and __exit__ or the contextlib module. It makes resource management deterministic and robust in the presence of exceptions.

37. ★ MUST ASK - Why is the with statement preferred for files and other resources?

Answer: with ensures the resource's cleanup protocol runs when the block exits, including when an exception occurs. This avoids duplicated cleanup code and reduces leaks such as forgetting to close a file.

38. ★ MUST ASK - How would you design exception handling in a production data-science script or model pipeline?

Answer: Validate inputs early, catch expected failures near clear boundaries, log useful context, preserve traceback information, avoid broad silent catches, and convert low-level failures into meaningful domain errors where appropriate. Fail fast on invalid prerequisites and recover only when a safe fallback exists.

39. ★ MUST ASK - When would you choose composition over inheritance?

Answer: Choose composition when a class needs capabilities from other components, when behavior may vary independently, or when inheritance would create a rigid hierarchy. Composition is often preferable for pipelines, model wrappers, data services, and interchangeable utilities.

40. ★ MUST ASK - How would you use OOP and exception handling together in a data-science project?

Answer: Encapsulate responsibilities in classes, define clear public methods, validate inputs at boundaries, raise domain-specific exceptions, and handle failures at the orchestration layer. For example, a DataLoader can raise DataLoadError, while a pipeline controller logs the problem and decides whether to retry or fail.

STANDARD QUESTIONS - OBJECT-ORIENTED PROGRAMMING

41. What is a class in Python?

Answer: A class is a user-defined type that bundles data and behavior. It defines the structure and operations available to instances.

42. What is an object?

Answer: An object is an instance of a class. It has identity, state represented by attributes, and behavior exposed by methods.

43. How do you instantiate a class?

Answer: Call the class: `obj = MyClass()`. Python creates an instance and runs its initialization process.

44. What is an instance?

Answer: An instance is a concrete object created from a class. Multiple instances can share the same class definition while holding different state.

45. What are attributes?

Answer: Attributes are names associated with an object or class, typically storing data or references to methods and other objects.

46. What are methods?

Answer: Methods are functions defined on a class. When retrieved through an instance, an instance method is bound to that instance.

47. What is the purpose of `__init__()`?

Answer: It initializes instance state after an object has been created. It is commonly used to validate and assign constructor arguments.

48. Can a class exist without `__init__()`?

Answer: Yes. If you do not define `__init__`, Python uses inherited initialization or the default behavior from `object` when appropriate.

49. What is `self`?

Answer: `self` is the conventional first parameter of an instance method and refers to the current instance.

50. Do you pass `self` explicitly when calling a method?

Answer: Normally no. `obj.method()` causes Python to bind `obj` to the method's first parameter. You only pass it explicitly when calling the function through the class, such as `Class.method(obj)`.

51. Can `self` be renamed?

Answer: Yes. `self` is a convention, not a keyword. Python's binding behavior depends on the position of the first parameter, not its name. You should still use `self` for readability.

52. What is the difference between an instance attribute and a class attribute?

Answer: An instance attribute is stored on a particular object; a class attribute is defined on the class and shared unless shadowed by an instance attribute.

53. What happens if an instance attribute has the same name as a class attribute?

Answer: Attribute lookup finds the instance attribute first, so the instance value shadows the class attribute for that object.

54. How do you access an attribute?

Answer: Use dot notation, such as `obj.name`. You can also use `getattr(obj, 'name')` for dynamic attribute access.

55. What is encapsulation?

Answer: Encapsulation means keeping related state and behavior together and limiting direct dependence on internal implementation details.

56. Does Python support true private variables?

Answer: Python does not enforce private instance variables at the language level. Leading underscores communicate intended non-public APIs, while double leading underscores trigger name mangling.

57. What does a single leading underscore mean?

Answer: It is a convention meaning the name is intended for internal use and is not part of the public API.

58. What does a double leading underscore mean?

Answer: For an instance/class attribute name like `__value`, Python applies name mangling, roughly changing it to `_ClassName__value` to reduce accidental clashes in subclasses.

59. What is abstraction?

Answer: Abstraction means exposing an interface and hiding unnecessary implementation details.

60. What is inheritance?

Answer: Inheritance allows a subclass to reuse or specialize members of a parent class.

61. What is single inheritance?

Answer: A subclass has one direct parent class, such as `class Child(Parent): ...`

62. What is multilevel inheritance?

Answer: A chain of inheritance, such as `Grandchild -> Child -> Parent`.

63. What is hierarchical inheritance?

Answer: Multiple subclasses inherit from the same parent class.

64. What is multiple inheritance?

Answer: A class directly inherits from two or more parent classes. It can enable mixins but requires careful design because of MRO.

65. What is method overriding?

Answer: A subclass defines a method with the same name as an inherited method to change or specialize its behavior.

66. Why use OOP in a data-science project?

Answer: OOP can isolate pipeline stages, configuration, model state, preprocessing logic, and services behind clear interfaces. It is most valuable when there is meaningful state or repeated behavior.

67. How would you design a reusable data preprocessing class?

Answer: Give it explicit configuration and state, such as feature columns and fitted transformers, and methods such as `fit`, `transform`, and `fit_transform`. Keep validation and transformation responsibilities focused.

68. How would you design a model wrapper class?

Answer: Expose a small interface such as `fit`, `predict`, `save`, and `load`, while hiding implementation details like preprocessing, model serialization, and validation where appropriate.

69. What is a property in Python?

Answer: A property lets attribute-style access invoke methods. `@property` commonly creates a read-only or computed attribute without changing the public calling style.

70. Why use a property instead of direct attribute access?

Answer: Properties can validate assignments, compute values dynamically, or preserve an API while changing internal implementation.

71. What are dunder methods?

Answer: Special methods with double underscores that integrate user-defined objects with Python's data model, such as `__len__`, `__eq__`, and `__iter__`.

72. What does `__str__()` control?

Answer: The human-readable string representation used by `str(obj)` and usually `print(obj)`.

73. What does `__repr__()` control?

Answer: The developer-oriented representation returned by `repr(obj)`, typically designed to be informative for debugging.

74. When does Python call `__str__()`?

Answer: Commonly when `str(obj)` is called or when `print(obj)` formats the object.

75. When does Python call `__repr__()`?

Answer: When `repr(obj)` is called, and often when objects are displayed in an interactive interpreter or container representations.

76. What is operator overloading?

Answer: Defining dunder methods so operators work meaningfully with custom objects, such as implementing `__add__` for `+`.

77. How would you overload `+` for a custom class?

Answer: Implement `__add__(self, other)` and decide what result is appropriate. Return a new object rather than mutating unexpectedly unless the API explicitly calls for mutation.

78. What is `__len__()`?

Answer: It defines length semantics for an object so `len(obj)` can call `obj.__len__()`.

79. What is `__eq__()`?

Answer: It defines custom equality behavior for `obj1 == obj2`. It should return a Boolean-like result and be consistent with the object's semantics.

80. What is object identity versus object equality?

Answer: Identity asks whether two references point to the same object (`is`); equality asks whether two objects compare as equivalent (`==`). Two distinct objects can be equal.

STANDARD QUESTIONS - ERROR HANDLING & EXCEPTIONS

81. What is an exception?

Answer: An exception is an object representing an abnormal runtime condition that interrupts the normal flow of execution.

82. What is the difference between an error and an exception in Python?

Answer: In everyday Python usage, 'error' is a broad term for something going wrong, while an exception is the runtime mechanism represented by an exception object. Syntax errors happen during parsing; many other failures are raised exceptions.

83. What is a syntax error?

Answer: A syntax error occurs when Python cannot parse the source according to the language grammar, such as a missing colon or unmatched parenthesis.

84. What is a runtime exception?

Answer: An exception raised while code is executing, such as `ValueError`, `TypeError`, or `KeyError`.

85. What is try/except?

Answer: A control structure for executing risky code and catching selected exceptions so the program can handle expected failures.

86. How do you catch multiple exception types?

Answer: Use a tuple, for example `except (ValueError, TypeError):`.

87. What is bare except?

Answer: `except:` catches every exception inheriting from `BaseException`, including control-flow exceptions such as `KeyboardInterrupt`. It is usually too broad.

88. Why should exceptions be caught narrowly?

Answer: Narrow catches document what you expect, prevent accidental suppression of unrelated bugs, and make debugging easier.

89. What is the purpose of except Exception?

Answer: It catches ordinary application exceptions while avoiding exceptions such as `KeyboardInterrupt` and `SystemExit`. It can be appropriate at a top-level boundary when paired with logging and deliberate recovery or shutdown.

90. What is else in try/except/else?

Answer: The `else` block executes only if the `try` block completed without raising an exception. It keeps success-only logic separate from code being monitored for errors.

91. What is finally?

Answer: `finally` is a cleanup block that executes when control leaves the `try` statement, whether the `try` succeeded or raised.

92. Can try have finally without except?

Answer: Yes. This is useful when cleanup is required even though the caller should still receive any exception.

93. Can try have else without except?

Answer: Yes, if there is another handler such as `except` or a `finally`? In Python syntax, an `else` clause may be combined with one or more `except` clauses, and it cannot stand alone without an accompanying `except`.

94. What is raise?

Answer: `raise` explicitly triggers an exception, either a new exception or the currently handled exception.

95. Can you raise a built-in exception?

Answer: Yes, for example `raise ValueError('invalid value')` or `raise TypeError('expected string')`.

96. Can you re-raise an exception?

Answer: Yes. Inside an `except` block, bare `raise` re-raises the current exception with its traceback.

97. What is a custom exception?

Answer: A user-defined exception class, usually derived from `Exception`, that represents an application-specific failure.

98. Why create custom exceptions?

Answer: They make errors more meaningful and allow callers to catch a precise domain-specific category.

99. What is the exception hierarchy?

Answer: Exceptions form an inheritance tree rooted at `BaseException`. `Exception` contains most ordinary application exceptions, with subclasses such as `ValueError`, `TypeError`, `KeyError`, and others.

100. Why is `KeyboardInterrupt` not usually caught by `except Exception`?

Answer: `KeyboardInterrupt` inherits directly from `BaseException`, not `Exception`, so `except Exception` does not normally intercept `Ctrl+C`.

101. What is a `ValueError`?

Answer: It means the type is appropriate but the value is invalid for the operation, such as `int('abc')`.

102. What is a `TypeError`?

Answer: It indicates an operation or function call received an inappropriate type or operand combination, such as adding an integer to a string.

103. What is a `KeyError`?

Answer: A `KeyError` occurs when code accesses a dictionary with a key that does not exist, for example `d['missing']`.

104. What is an `IndexError`?

Answer: An `IndexError` occurs when sequence indexing uses a position outside the valid range.

105. What is a `ZeroDivisionError`?

Answer: It occurs when Python attempts division or modulus by zero for numeric types where zero is invalid as a divisor.

STANDARD QUESTIONS - PRACTICAL, DEBUGGING & BEST PRACTICES

106. How should exceptions be logged in production code?

Answer: Use structured logging with useful context such as operation, record identifier, stage, exception type, and traceback where appropriate. Avoid sensitive data such as credentials, tokens, or personal data.

107. Should you use `print()` for exception logging?

Answer: For production applications, prefer the logging module because it supports levels, formatting, handlers, timestamps, centralized collection, and structured operational practices.

108. How do you preserve the original traceback?

Answer: Use bare `raise` when re-raising from an `except` block, and use exception chaining with `raise ... from e` when translating an error. Avoid unnecessarily replacing the original cause.

109. What is exception chaining?

Answer: It is Python's mechanism for recording that one exception caused another. Explicit chaining uses `raise NewError(...) from e`; implicit chaining also occurs when a new exception is raised while handling another.

110. What does `raise NewError() from e` mean?

Answer: It raises `NewError` and explicitly sets `e` as its cause, making the relationship visible in the traceback.

111. What does `raise NewError() from None` mean?

Answer: It suppresses the display of the automatically chained context while still raising the new exception. Use it when the lower-level context would add noise and the abstraction boundary is deliberate.

112. Why avoid catching `Exception` and doing nothing?

Answer: It silently discards failures, making data corruption, incomplete processing, and debugging much harder.

113. What is exception swallowing?

Answer: Catching an exception and failing to report, propagate, recover, or otherwise account for it. It is dangerous because failures become invisible.

114. When should you convert one exception type into another?

Answer: At an abstraction boundary. For example, a database client's low-level exception can be wrapped as `DataLoadError` so the rest of the pipeline does not depend on vendor-specific details.

115. How do you validate input before a risky operation?

Answer: Check preconditions that are cheap and meaningful, such as required keys, ranges, file existence when appropriate, schema, or configuration. Still handle race conditions and other runtime failures that validation cannot eliminate.

116. When is EAFP preferred in Python?

Answer: EAFP ('Easier to Ask Forgiveness than Permission') means attempting the operation and handling the expected exception. It is useful when the operation itself is the most reliable test and failures are relatively uncommon.

117. When is LBYL preferred?

Answer: LBYL ('Look Before You Leap') means checking conditions before acting. It can be clearer when validation is cheap, the check has independent business meaning, or repeated exceptions would be noisy.

118. What are the trade-offs between EAFP and LBYL?

Answer: EAFP often produces simpler code and avoids check-then-act race conditions. LBYL can make expected validation paths explicit, but checks can become duplicated or stale. Choose based on semantics and failure frequency.

119. How should you handle a missing key in a dictionary?

Answer: Use `dict.get()` when absence has a normal default, `in` when you need explicit presence logic, or catch `KeyError` when missing data is exceptional and tied to an operation.

120. How should you handle file errors?

Answer: Use `with open(...)` for cleanup and catch relevant exceptions such as `FileNotFoundError`, `PermissionError`, or `OSError` where the program can respond meaningfully.

121. How should you handle user input conversion errors?

Answer: Wrap the conversion in a narrow `try` block and catch `ValueError`, then show a useful validation message or ask for corrected input.

122. What is a context manager?

Answer: An object that manages setup and cleanup around a block of code through the context management protocol, typically `__enter__` and `__exit__`.

123. Why is `with open(...)` preferred?

Answer: It guarantees file cleanup when the block exits, including when an exception is raised, without requiring a manual `close()` in every branch.

124. What is `__enter__()`?

Answer: It performs context setup and returns the value bound after as in a `with` statement.

125. What is `__exit__()`?

Answer: It performs cleanup when the context ends and receives exception information if the body raised. It can optionally suppress the exception by returning a true value.

126. Can `__exit__()` suppress an exception?

Answer: Yes. If `__exit__()` returns a truthy value, Python treats the exception as handled. This should be done only deliberately because suppression can hide failures.

127. How do exceptions interact with `finally`?

Answer: Before an exception or return leaves the `try` construct, Python executes `finally`. Cleanup runs first, then the pending control flow continues unless `finally` changes it.

128. What happens if `finally` itself raises an exception?

Answer: The new exception propagates and can replace the earlier exception for normal propagation purposes. This is another reason to keep `finally` simple and reliable.

129. What happens if both `try` and `finally` return?

Answer: The `finally` return wins. It can override the `try` return and even suppress an exception, so returning from `finally` is generally discouraged.

130. How should exception handling be designed around ML model loading?

Answer: Separate validation from loading, catch missing files/configuration/dependency problems at a clear boundary, preserve the original cause, log context, and expose a meaningful higher-level error.

131. How should exceptions be designed in a data pipeline?

Answer: Separate data-validation errors, transient infrastructure failures, and programming defects. Recover only from failures with a safe recovery strategy; otherwise fail with diagnostics and clear stage information.

132. How do you avoid overusing exceptions as normal control flow?

Answer: Use exceptions for genuinely exceptional or failure-oriented conditions. Prefer ordinary conditionals for expected branching when they make intent clearer.

133. What is graceful degradation?

Answer: Providing a safe reduced-functionality fallback when a non-critical component fails, such as using cached data when an optional external service is unavailable.

134. What is fail fast?

Answer: Detect invalid prerequisites as early as possible and stop before expensive or unsafe work continues. It prevents later, less understandable failures.

135. How would you test exception handling?

Answer: Write tests that intentionally trigger expected exceptions and verify exception type, relevant message/data, system state, logging or recovery behavior, and that unrelated exceptions are not swallowed.

DAY 4 RAPID REVISION

OOP	class vs object; instance/class attributes; <code>__init__</code> ; <code>self</code> ; encapsulation; inheritance; polymorphism; abstraction; over
Exceptions	syntax vs runtime errors; <code>try/except</code> ; specific catches; <code>else</code> ; <code>finally</code> ; <code>raise</code> ; re-raise; custom exceptions; hierarchy; cha
Production	logging; EAFP vs LBYL; domain-specific exceptions; validation; cleanup; fail-fast; graceful degradation; testing failur

Mastery test: You have mastered a question when you can answer it without notes, write a small example, explain why the behavior occurs, and describe when you would and would not use the technique in a real data-science or AI project.

Interview habit: Prefer precise language. Explain the concept first, then give a small example, then add the practical trade-off. For exceptions, distinguish expected failures from programming defects and avoid broad silent catches.